

Spring Framework — Day 16 Notes

Singleton Design Pattern (continued) & Bean Scopes in Spring IoC

In the previous session, we learned about the Singleton design pattern — how to create a class as a singleton, and in which situations we should design a class this way.

1. When to Use the Singleton Pattern — Case 2

Rule: If a class has member variables whose values are fixed (constant) and do not change across objects, that class is a good candidate to be made a singleton.

The reasoning is simple: if the value never changes, then creating multiple objects of the class would only create multiple copies holding the exact same value. That is wasted memory, so it is better to allow only one object to exist.

```
class Cylinder {  
  
    private static final float pi = 3.14f;  
  
    private Cylinder() {  
    }  
  
    public static void getArea(int radius) {  
        pi * radius * radius;  
    }  
}
```

Here, **pi** is a fixed value. Even if many objects of the **Cylinder** class were created, every object would hold the same value of pi. So instead of allowing multiple objects, it is better to design this class as a singleton.

2. Bean Scope in Spring

Definition: Bean scope tells the IoC (Inversion of Control) container how many objects should be created for a given bean definition.

By default, the scope of a bean defined in XML is **singleton**. We can change this default behaviour by adding the **scope** attribute at the bean level:

```
scope="<scope-name>"
```

Spring provides four types of bean scope:

- **singleton** — only one object is created for the bean definition (default).
- **prototype** — a new object is created every time the bean is requested.
- **request** — a new object is created for every HTTP request (used only in web applications).
- **session** — a new object is created for every HTTP session (used only in web applications).

Note: The **request** and **session** scopes are relevant only when working inside a web application, since they depend on the HTTP request/session lifecycle.

Let us look at how each scope behaves, and what output it produces.

3. Singleton Scope — Demonstration

Step 1 — Create a simple class:

```
public class A {  
    public A() {  
    }  
}
```

Step 2 — Configure the bean in the XML file with scope "singleton":

```
<?xml version="1.0" encoding="UTF-8"?>  
<beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="  
        http://www.springframework.org/schema/beans  
        https://www.springframework.org/schema/beans/spring-beans.xsd">  
  
    <bean id="a" class="com.day16.A" scope="singleton">  
    </bean>  
  
</beans>
```

Step 3 — Test it from the Main class:

```
public class Main {  
    public static void main(String[] args) {  
        ApplicationContext context =  
            new ClassPathXmlApplicationContext("com/day16/application-context.xml");  
  
        A bean1 = context.getBean("a", A.class);  
        A bean2 = context.getBean("a", A.class);  
  
        System.out.println("Is the hashCode same ?");  
        System.out.println("Ans : " + (bean1.hashCode() == bean2.hashCode()));  
    };  
  
    System.out.println(bean1.hashCode());  
    System.out.println(bean2.hashCode());  
}
```

Since the scope is singleton, the container returns the **same object** every time **getBean()** is called. So both hash codes will be equal.

4. Prototype Scope — Demonstration

If we change the bean scope in the same XML file from **singleton** to **prototype**, and run the same code in the Main class, we get a different result.

What changes in the XML:

```
From:
<bean id="a" class="com.day16.A" scope="singleton">
</bean>

To:
<bean id="a" class="com.day16.A" scope="prototype">
</bean>
```

Output:

```
Is the hashCode same ?
Ans : false
1106131243
510109769
```

With prototype scope, every call to **getBean()** creates a **new object**. That is why the two hash codes are different.

5. Choosing the Right Scope for a Class

Classes that are usually kept as prototype:

- All domain classes
- Java beans (POJOs that hold data)
- DAO classes (Data Access Object)
- TO / DTO classes (Transfer Object)

These classes typically represent data that changes for every request or every record, so a new object is needed each time.

Classes that are usually kept as singleton:

- All component classes
- Business logic classes
- Service classes

These classes usually contain logic/behaviour rather than request-specific data, so one shared object is enough.

Example — Why a Servlet-handled request class needs prototype scope:

Suppose a servlet receives request data such as Username, Email, and Password:

```
class UserServicelet extends HttpServlet {  
  
    public void doPost(HttpServletRequest req, HttpServletResponse res) {  
        // handle the incoming request  
    }  
  
}
```

Every new HTTP request following the HTTP protocol carries different data. So for every incoming request, we need a new object to hold that request's data — which is exactly why we use **prototype** scope for such classes.

6. How Many Ways Can We Break a Singleton Class?

A singleton class is designed to allow only one object. However, this guarantee can be broken. There are two common ways to break a singleton class:

- By using multiple Threads
- By using the Reflection API

6.1 — Breaking Singleton Using Threads

Consider the following singleton class. Notice that a delay (**Thread.sleep**) is added inside the object-creation logic purely to demonstrate the problem:

```
class SingleTonProgram {  
  
    private static SingleTonProgram ref = null;  
  
    private SingleTonProgram() {  
    }  
  
    public static SingleTonProgram getObject() {  
        if (ref == null) {  
            System.out.println("Wait, I will create the object...");  
            try {  
                Thread.sleep(5000);  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
            ref = new SingleTonProgram();  
        }  
        return ref;  
    }  
  
}
```

Calling it from two threads inside the main method:

```

Runnable r1 = () -> {
    SingletonProgram ref = SingletonProgram.getObject();
    System.out.println(Thread.currentThread().getName());
    System.out.println(ref.hashCode());
};

Thread t1 = new Thread(r1);
Thread t2 = new Thread(r1);

t1.start();
t2.start();

```

Problem: Because of the delay, both threads can enter the **if (ref == null)** block before either of them finishes creating the object. As a result, two different objects get created — the singleton guarantee is broken.

Note: Solution: Mark the **getObject()** method as **synchronized**. This ensures only one thread can execute the object-creation logic at a time, so only one object is ever created.

6.2 — Breaking Singleton Using the Reflection API

Even a private constructor does not fully protect a singleton class. Using Java's Reflection API, we can access the private constructor directly and create another object:

```

SingletonProgram ref1 = SingletonProgram.getObject();
System.out.println(ref1.hashCode());

try {
    Constructor<SingletonProgram> constructor =
        SingletonProgram.class.getDeclaredConstructor();
    // finds the constructor, even if it is private

    constructor.setAccessible(true);
    // ignores the private keyword and allows using this constructor

    SingletonProgram ref2 = constructor.newInstance();
    System.out.println(ref2.hashCode());

} catch (NoSuchMethodException e) {
    e.printStackTrace();
} catch (SecurityException e) {
    e.printStackTrace();
} catch (InstantiationException e) {
    e.printStackTrace();
} catch (IllegalAccessException e) {
    e.printStackTrace();
} catch (IllegalArgumentException e) {
    e.printStackTrace();
} catch (InvocationTargetException e) {
    e.printStackTrace();
}

```

getDeclaredConstructor() locates the constructor even though it is private. **setAccessible(true)** then bypasses the private access modifier, allowing **constructor.newInstance()** to create a fresh object —

ref2 — which will be different from **ref1**. This again breaks the singleton guarantee.

End of Day 16 Notes